

**Concordia University**  
**COMP 5461 – Operating Systems**

**Summer 2026**

**Programming assignment 1**

---

|                         |                                                                                                                       |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Deadline:</b>        | Tuesday, June 9 2026.                                                                                                 |
| <b>Late Submission:</b> | No late submission.                                                                                                   |
| <b>Teams:</b>           | The assignment can be done individually or in teams of 2.                                                             |
| <b>Purpose:</b>         | The purpose of this assignment is to apply in practice the multi-threading features of the Java programming language. |

---

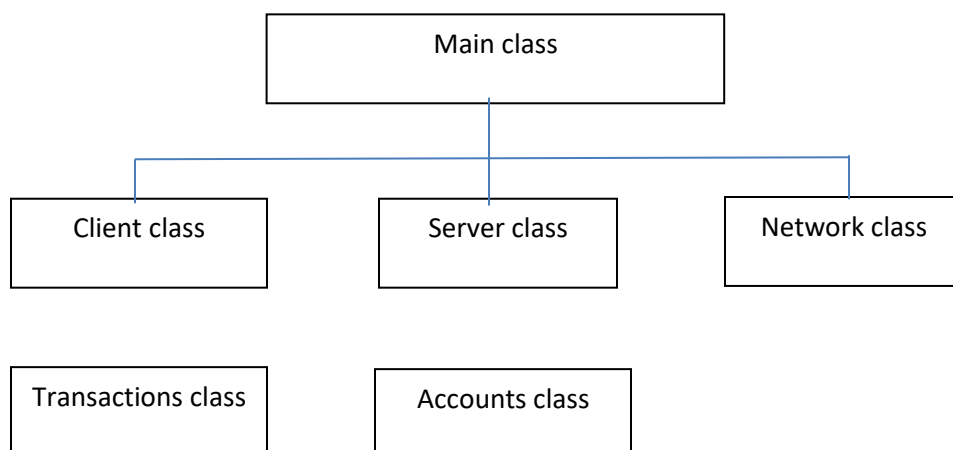
- **Problem specification.**

In a client-server system, the client application sends requests to a server application through a network connection. In such system, the user interface is implemented in the client and the database is stored in the server.

You are required to implement a client-server application to process banking transactions such as withdrawals and deposits. In modern banking systems, a costumer accesses a bank account using an access card at an ATM, at the counter or on the web.

- **Implementation.**

The following diagram illustrates the classes for the client-server banking application.



- Network class.

The Network class provides the infrastructure to allow the client and the server to process the transactions. The client and the server need to be connected (using connect()) to the network prior to an exchange. The Network class also implements an input buffer (inComingPacket[]) and an output buffer (outGoingPacket[]) to respectively receive transactions from the client and to return updated transactions to the client. The capacity of these buffers are 10 elements, so the network indicates whether they are full or empty.

- Client class.

The Client class reads all the transactions from a file (transaction.txt) and saves them in an array (transaction[]). A transaction is implemented by the Transactions.class.

Using the send() method of Network class, the client transfers the transactions to the network input buffer and it yields the cpu in case the network input buffer is full.

Also, using the receive() method of Network class, the client retrieves the updated transactions from the network output buffer and yields the cpu in case the buffer is empty. Each updated transaction received is displayed immediately on the console.

- Server class.

The Server class reads all the accounts from a file (account.txt) and saves them in an array (account[]). An account is implemented by the Accounts class. Using the transferrIn() method of Network class, the server retrieves the transactions from the network input buffer and performs the operations (withdraw, deposit, query) on the specific accounts. It yields the cpu in case the buffer is empty.

Each updated transaction is transmitted to the network output buffer using the transferOut( ) method of Network class and the server yields the cpu in case the buffer is full.

- **Problems.**

- You need to complete the Java program that is provided by implementing 4 threads so that the client, the server and the network all run concurrently. The client has 2 threads, one for sending the transactions and another for receiving the completed transactions.  
In case the input and output network buffers are full or empty, each client or server thread must yield the cpu first using busy-waiting and second using the Java method Thread.yield(). The network thread executes an infinite loop that ends when both client and server threads have disconnected. In case the client or sever threads are still connected the network thread must continuously yield the cpu.

- You must record the running times of both client threads and the server thread using the Java method `System.currentTimeMillis()`.
    - First set-up the Java program to run with busy-waiting and second using the Java method `Thread.yield()`.
    - Comment on the running times when using busy-waiting and yield. Explain whether busy-waiting or/and yield are beneficial for the overall performance of the system or of the individual threads.
  - You need to provide output test cases with the appropriate running times for the client and the server threads. Perform 3 different runs of the program and explain why there is a difference in the running times.
- **Sample output test cases.**
    - See attached text files.
  - **Evaluation.**

You will be evaluated mostly on the implementation of the required methods, the implementation of the threads, the measurements of the running times and the voluntary sharing of the cpu by the threads.

- Evaluation criteria

| Criteria                                                | Marks |
|---------------------------------------------------------|-------|
| Implementation of the 4 threads                         | 35%   |
| Implementation of the main class                        | 15%   |
| Answer to a question during the demo                    | 10%   |
| Implementation of the <code>yield()</code> method       | 10%   |
| Implementation of the measurements of the running times | 10%   |
| Output test cases including running times               | 20%   |

- **Required documents.**
  - Source codes in Java.
  - Output test cases.
  - I have included DEBUG flags in the source code in order to help you trace the program but once your program works properly you should put the DEBUG flags in comments.
- **Submission.**

- Create one zip file, containing the necessary files (.java, .txt and test cases). If the assignment is done individually, your file should be called *pa1\_studentID*, where *pa1* is the number of the assignment and *studentID* is your student ID number. If the work is done in a team of 2 people, the zip file should be called *pa1\_studentID1\_studentID2* where *studentID1* and *studentID2* are the student ID numbers of each student.
- Upload your zip file on moodle under *Programming Assignment 1* before the due date.

- **IMPORTANT (Please read very carefully):**

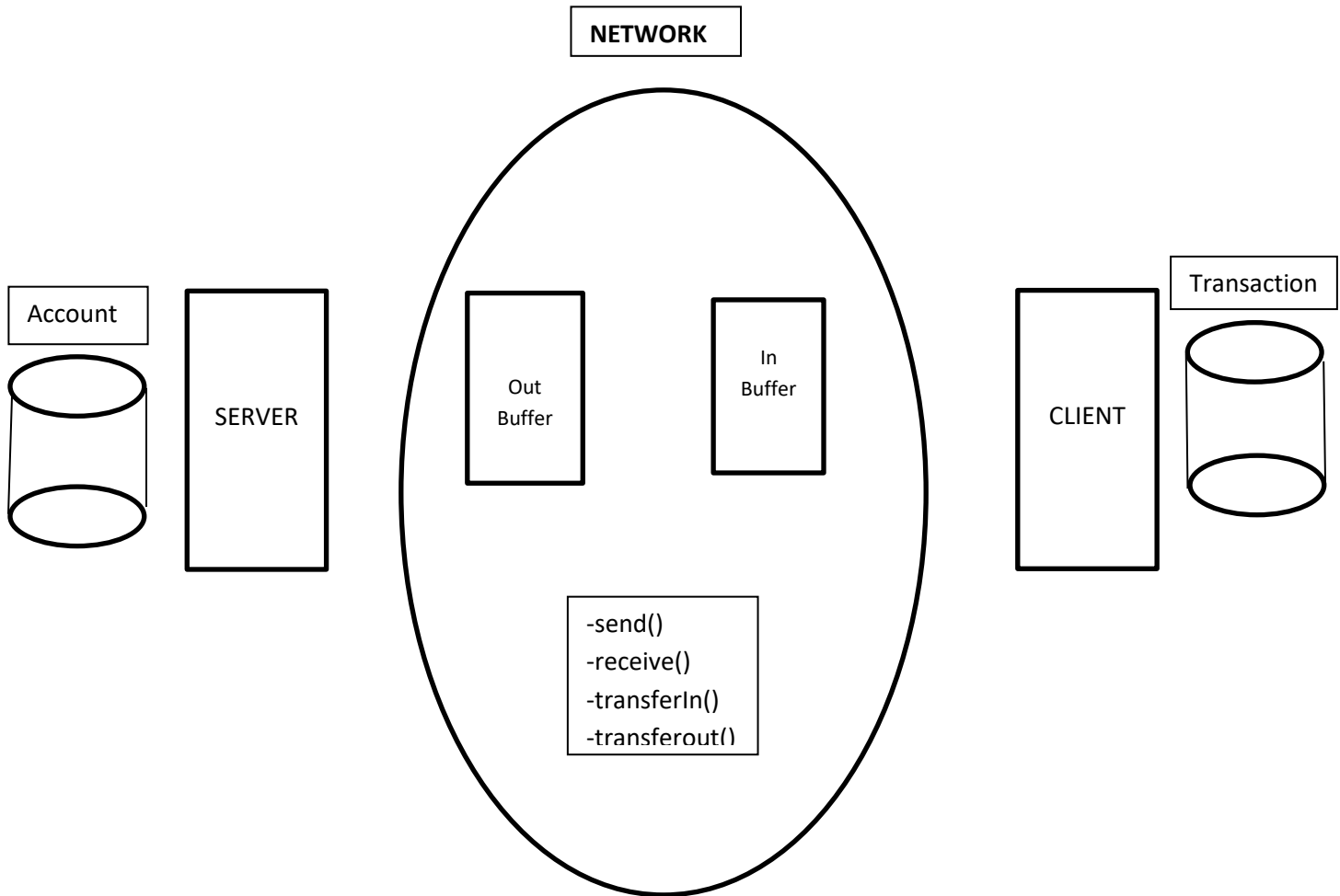
A demo will take place with the markers afterwards. Markers will inform you about the details of demo time and how to book a time slot for your demo. If working in a group, both members must be present during demo time. Different marks may be assigned to teammates based on this demo.

- If you fail to demo, a zero mark is assigned regardless of your submission.
- If you book a demo time, and do not show up, for whatever reason, you will be allowed to reschedule a second demo but a penalty of 50% will be applied.
- Failing to demo at the second appointment will result in zero marks and no more chances will be given under any conditions.

- **Disclaimer.**

- Note that this code has been tested on Windows. You may need to make changes if you would like to run on other OS. However, you are not permitted to change the implementation of the source code provided nor change the data types and sizes but you should only implement the features where specified.

- Detailed implementation of the classes



#### ▪ Client class

| Client                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>- numberOfTransactions : int /* total number of transactions to process */</li> <li>- maxNbTransactions : int /* maximum number of transactions */</li> <li>- transactions : Transactions[ ] /* transaction array */</li> <li>- clientOperation : String /* sending, receiving */</li> <li>- objNetwork : Network /* handle to access network methods */</li> </ul> |
| Client(String operation)<br>+getNumberOfTransactions() : int<br>+getClientOperation() : String<br>+setNumberOfTransactions(int nbOfTrans) : void<br>+setClientOperation(String operation) : void<br>+readTransactions() : void<br>+sendTransactions() : void<br>+receiveTransactions(Transactions transact) : void<br>+toString() : String<br>+run() : void                                                |

▪

▪ Transactions class

| Transactions                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>- accountNumber : String /* account number */</li> <li>- operationType : String /* deposit, withdrawal, query */</li> <li>- transactionAmount : double /* transaction amount */</li> <li>- transactionBalance : double /* updated account balance */</li> <li>- transactionError : String /* NSF, invalid amount or account, none */</li> <li>- transactionStatus : String /* pending, sent, received, done */</li> </ul>                                                                                                                                                                                       |
| <p>Transactions( )</p> <p>+ getTransactionType( ) : String</p> <p>+ getAccountNumber( ) : String</p> <p>+ getTransactionAmount( ) : double</p> <p>+ getTransactionBalance( ) : double</p> <p>+ getTransactionError( ) : String</p> <p>+ getTransactionStatus( ) : String</p> <p>+ setAccountNumber(String accNumber ) : void</p> <p>+ setTransactionType(String opType) : void</p> <p>+ setTransactionAmount(double transAmount ) : void</p> <p>+ setTransactionBalance(double transBalance ) : void</p> <p>+ setTransactionError(String transError) : void</p> <p>+ setTransactionStatus(String transStatus) : void</p> <p>+ toString( ) : String</p> |

▪ Server class

| Server                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>- numberOfTransactions : int /* total number of transactions received */</li> <li>- numberOfAccounts : int /* total number of accounts saved */</li> <li>- maxNbAccounts : int /* maximum number of accounts */</li> <li>- transaction : Transactions /* a transaction to process */</li> <li>- objNetwork : Network /* handle to access network methods */</li> <li>- account : Accounts[] /* account array */</li> </ul>                                                                                                                                            |
| <p>Server( )</p> <p>+getNumberOfTransactions( ) : int</p> <p>+getNumberOfAccounts( ) : int</p> <p>+getMaxNbAccounts( ) : int</p> <p>+setNumberOfTransactions(int nbOfTrans) : void</p> <p>+setNumberOfAccounts(int nbOfAcc) : void</p> <p>+setMaxNbAccounts(int nbOfAcc) : void</p> <p>+initializeAccounts( ) : void</p> <p>+findAccount(String accNumber) : int</p> <p>+processTransactions(Transaction trans) : boolean</p> <p>+deposit(int i, double amount) : double</p> <p>+withdraw(int i, double amount) : double</p> <p>+query(int i) : double</p> <p>+toString( ) : String</p> <p>+run() : void</p> |

▪ Accounts class

| Accounts |                                                       |
|----------|-------------------------------------------------------|
| -        | accountNumber : String /* unique account number */    |
| -        | accountType : String /* chequing, saving, credit */   |
| -        | firstName : String /* first name of account holder */ |
| -        | lastName : String /* last name of account holder */   |
| -        | balance : double /* account balance */                |
| +        | getAccountNumber( ) : String                          |
| +        | getAccountType( ) : String                            |
| +        | getFirstName( ) : String                              |
| +        | getLastname( ) : String                               |
| +        | getBalance( ) : double                                |
| +        | setAccountNumber(double accNumber) : void             |
| +        | setAccountType(String accType) : void                 |
| +        | setFirstName(String fName) : void                     |
| +        | setLastname(String lName) : void                      |
| +        | setBalance(double bal) : void                         |
| +        | toString( ) : String                                  |

▪ Network class

| Network |                                                                                                            |
|---------|------------------------------------------------------------------------------------------------------------|
| -       | clientIP : string /* IP of client application */                                                           |
| -       | serverIP : string /* IP of server application */                                                           |
| -       | portID : int /* port ID of client application */                                                           |
| -       | clientConnectionStatus : String /* connected, disconnected */                                              |
| -       | serverConnectionStatus : String /* connected, disconnected */                                              |
| -       | maxNbPackets : int /* capacity of network buffers */                                                       |
| -       | inComingPacket : Transactions[10] /* network input buffer */                                               |
| -       | outGoingPacket : Transactions[10] /* network output buffer */                                              |
| -       | inBufferStatus, outBufferStatus : String /* normal, full, empty */                                         |
| -       | inputIndexClient, inputIndexServer, outputIndexServer, outputIndexClient : int /* buffer index position */ |
| -       | networkStatus : String /* active, inactive */                                                              |
|         | Network(String context)                                                                                    |
| +       | getClientIP( ) : String                                                                                    |
| +       | getServerIP( ) : String                                                                                    |
| +       | getPortID( ) : integer                                                                                     |
| +       | getClientConnectionStatus( ) : String                                                                      |
| +       | getServerConnectionStatus( ) : String                                                                      |
| +       | getInBufferStatus( ) : string                                                                              |
| +       | getOutBufferStatus( ) : string                                                                             |
| +       | getNetworkStatus( ) : String                                                                               |
| +       | getInputIndexClient( ) : int                                                                               |
| +       | getInputIndexServer( ) : int                                                                               |
| +       | getIOutputIndexClient( ) : int                                                                             |
| +       | getOutputIndexServer( ) : int                                                                              |
| +       | setClientIP(String cip) : void                                                                             |
| +       | setServerIP(String sip) : void                                                                             |

```
+ setPortID(int pid) : void
+ setClientConnectionStatus(String connectStatus) : void
+ setServerConnectionStatus(String connectStatus) : void
+ setNetworkStatus(String netStatus ) : void
+ setInBufferStatus(String inBufStatus) : void
+ setOutBufferStatus(String outBufStatus) : void
+ setInputIndexClient(int i1) : void
+ setInputIndexServer(int i2) : void
+ setOutputIndexClient(int o2) : void
+ setOutputIndexServer(int o1) : void
+ connect(String IP) : boolean
+ disconnect(String IP) : boolean
+ send(Transactions inPacket ) : boolean
+ receive(Transactions outPacket) : boolean
+ transferOut(Transactions outPacket ) : boolean
+ transferIn(Transactions inPacket) : boolean
+ toString() : String
+run() : void
```